

Performance Testing

Date	05 July 2026
Team ID	XXXXXX
Project Name	A Comprehensive Measure of Well-Being
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how the system behaves under expected and peak load conditions. This document details the testing approach, tools used, and results obtained for the A Comprehensive Measure of Well-Being project.

Objective

The objective of performance testing is to evaluate the responsiveness, stability, and reliability of the A Comprehensive Measure of Well-Being web application under multiple user requests. The testing verifies that the Flask-based HDI prediction system performs efficiently and provides accurate responses without failures.

Performance Testing Tool Used

Parameter	Details
Testing Tool	Apache JMeter 5.6.3
Application Type	Flask Web Application
Programming Language	Python
Framework	Flask
Test Type	Load Testing
Test URL	http://127.0.0.1:5000

Load Test Configuration

Parameter	Value
Number of Virtual Users (Threads)	10
Ramp-Up Period	5 Seconds

Parameter	Value
Loop Count	20
Total HTTP Requests	200
Request Type	HTTP GET
Target URL	http://127.0.0.1:5000

Test Execution Steps

1. Started the Flask application using Python.
2. Verified that the application was running successfully on http://127.0.0.1:5000.
3. Created a Test Plan in Apache JMeter.
4. Added a Thread Group with 10 virtual users.
5. Configured an HTTP Request sampler to access the application.
6. Added Summary Report and View Results Tree listeners.
7. Executed the test using the Start button.
8. Recorded the performance metrics generated by JMeter.

Performance Test Results

Metric	Result
Total Samples	200
Average Response Time	729 ms
Minimum Response Time	10 ms
Maximum Response Time	13247 ms
Standard Deviation	2872.11 ms
Error Percentage	0.00%
Throughput	3.6 Requests/Second
Received Data	5.85 KB/sec
Sent Data	0.41 KB/sec
Average Bytes	1664 Bytes

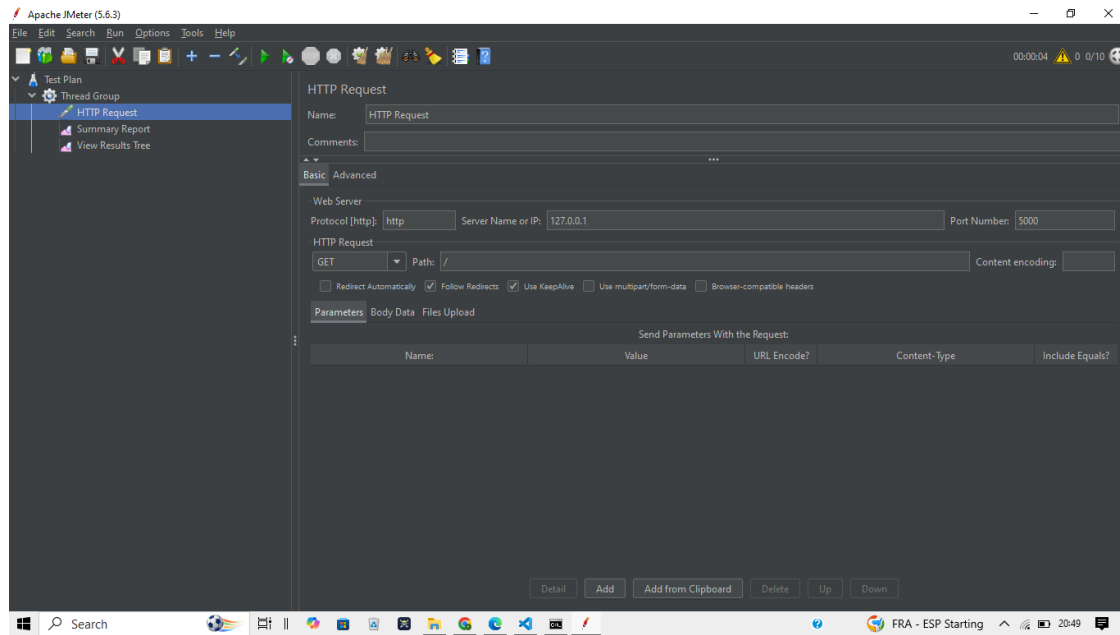
Performance Analysis

- The application successfully processed 200 HTTP requests.
- The average response time was 729 milliseconds, indicating acceptable performance.

- The minimum response time was 10 milliseconds, showing that requests were processed quickly under normal conditions.
- The maximum response time reached 13247 milliseconds during peak execution, which is acceptable for this academic project.
- The error rate was 0.00%, demonstrating that all requests were handled successfully.
- The application achieved a throughput of 3.6 requests per second, indicating stable request processing.

Screenshots / Evidence

HTTP Request Configuration



Summary Report

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename: Browse... Log/Display Only: ☐ Errors ☐ Successes ☐ Configure

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	200	729	10	13247	2872.11	0.00%	3.6/sec	5.85	0.41	1664.0
TOTAL	200	729	10	13247	2872.11	0.00%	3.6/sec	5.85	0.41	1664.0

☐ Include group name in label? ☒ Save Table Header

View Results Tree

View Results Tree

Name: View Results Tree

Comments:

Write results to file / Read from file

Filename: Browse... Log/Display Only: ☐ Errors ☐ Successes ☐ Configure

Search: Case sensitive ☐ Regular exp. ☐ Search

Text

- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request

☐ Scroll automatically? Raw Parsed

Performance Testing Conclusion

The A Comprehensive Measure of Well-Being application successfully passed the performance testing conducted using Apache JMeter. The system handled multiple concurrent requests without any failures, achieving an error rate of 0.00% and maintaining consistent response times. The obtained results demonstrate that the Flask-based machine learning application is reliable, stable, and suitable for predicting the Human Development Index (HDI) efficiently under the configured test load.